

VLSI Practice 08 Report

2022171005 권민찬

1. Architecture Design

i. Optimization Using CSD

Canonical Signed Digit을 이용하면 연속되는 1들을 1과 -1로 표현하여 어떤 2진수 숫자에서 1을 최대한 줄일 수 있다. 이를 이용하면 곱셈기로 구현해야 할 연산을 shifter과 adder만으로 구현할 수 있게 되어 회로의 면적을 줄일 수 있다. FIR 필터에서는 현재의 입력이 시간이 지나도 계속 쓰이기 때문에 computation sharing 기법을 이용하여 연산 자원을 더 아낄 수도 있다. 5개의 coefficient 사이에서 공통되는 비트 패턴을 발견할 수 있다면, 하나의 tap에서 수행된 연산의 결과를 다른 tap에서도 이용할 수 있고, 역시 연산 자원을 적게 사용할 수 있다.

계수 최적화의 목표는 단순히 1의 개수를 줄이는 것뿐만 아니라, 반복되는 비트패턴을 최대한 많이 만들고 발견해내는 것이다. 원래 CSD의 원칙은 이웃하는 1 혹은 -1은 이웃하면 안된다는 것인데, 비트 패턴을 많이 만들기 위해 이 원칙을 무시하고 최적화를 진행했다. Transposed Form에서는 모든 계수가 현재의 x와 먼저 곱해지고 지연이 일어나기 때문에 하나의 계수 내에서의 (per-tap) 비트패턴만을 이용할 수 있다. 반면 Direct form에서는 각 계수가 한 탭씩 지연된 입력 x에 곱해지기 때문에 여러 계수가 모여 (inter-tap) 생긴 비트패턴을 이용할 수 있기 때문에 최적의 구조가 만들어질 가능성이 높다.

C0	0	0	1	0	1	0	-1	0	0	0	0	1	0	-1	x[n]
C1	0	-1	0	1	0	-1	0	-1	0	-1	0	1	0	0	x[n-1]
C2	0	1	0	0	0	0	1	0	0	0	0	-1	0	1	x[n-2]
C3	1	1	0	1	-1	0	1	0	0	0	0	0	0	-1	x[n-3]
C4	1	0	-1	0	1	0	0	-1	0	1	-1	0	0	-1	x[n-4]

[Direct Form에서의 최적화 시도]

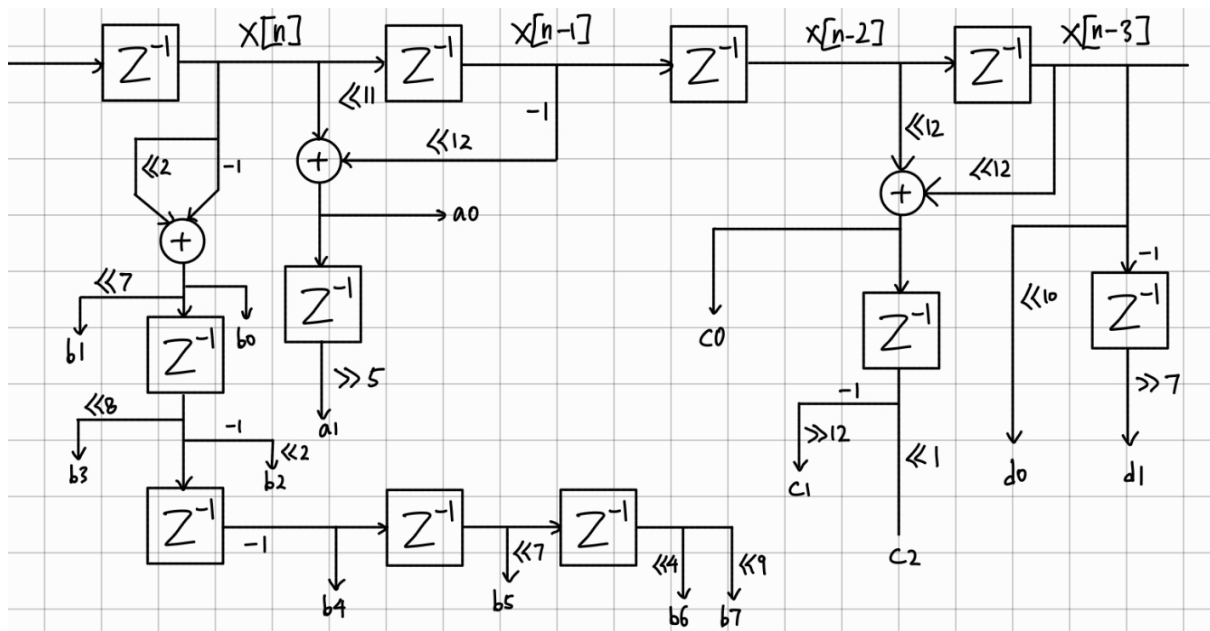
C0	0	0	1	0	1	0	-1	0	0	0	0	1	0	-1	x[n]
C1	0	-1	0	1	0	-1	0	-1	0	-1	0	1	0	0	x[n]
C2	0	1	0	0	0	0	1	0	0	0	0	-1	0	1	x[n]
C3	0	-1	0	0	1	0	1	0	0	0	0	0	0	-1	x[n]
C4	1	0	-1	0	1	0	0	-1	0	0	1	0	0	-1	x[n]

[Transposed Form에서의 최적화 시도]

위의 표처럼 최적화를 해볼 수 있다. Direct Form에서는 4개의 공통 비트패턴을 발견할

수 있었다. 4개의 비트패턴을 만들기 위해 필요한 가산기는 3개, 비트패턴 15개를 전부 더하기 위해 필요한 가산기는 14개로 총 17개의 가산기가 필요하다. Transposed Form에서도 4개의 공통 비트패턴을 만들기 위해 3개의 가산기가 필요하고, 비트패턴 15개를 더해주기 위해 14개의 가산기가 필요하므로 총 17개의 가산기가 필요하다.

ii. Optimized Filter Architecture



[Optimized Direct Form Architecture]

$$A = X[n] \ll 11 - X[n-1] \ll 12$$

$$B = X[n] \ll 2 - X[n]$$

$$C = X[n-2] \ll 12 + X[n-3] \ll 12$$

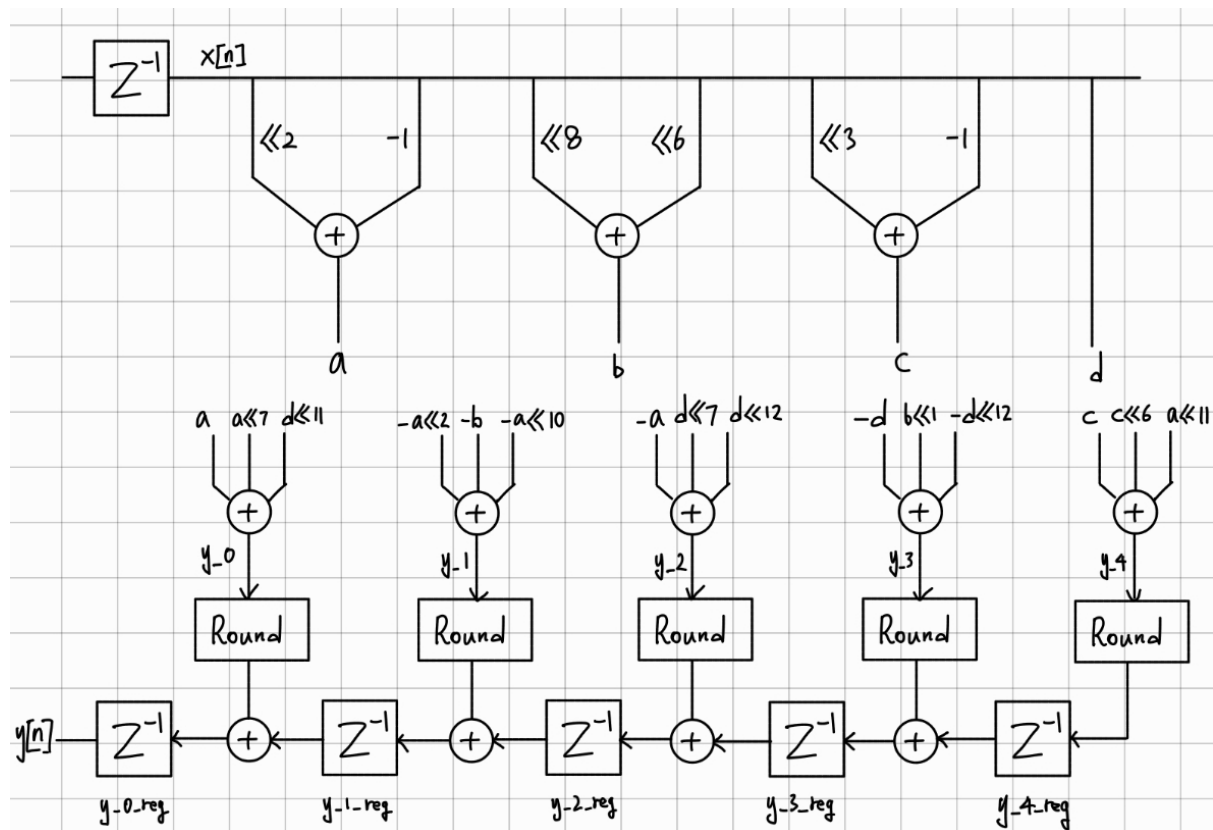
$$D = X[n-3]$$

위의 4가지 비트패턴을 만들기 위해 가산기가 3개 이용됐고, 이 연산 결과가 필요한 시점까지 지연시키기 위한 레지스터들과 필요한 shift가 위의 회로도에 나와있다. 그렇게 해서 생긴 결과 a0, ... d1까지 전부 가산기를 이용해서 더하면 최종결과가 출력된다.

Direct Form을 구현하기 위해서는 공통 비트패턴을 만든 뒤 그 비트패턴이 더해져야 할 시점까지 레지스터를 이용해 지연을 시켜야 하기 때문에 기존에 존재하던 Delay line을 제외하고도 더 많은 레지스터가 필요하다. 이는 면적을 늘리는 원인이 될 수 있다. 또한 Direct Form을 이용하면 round unit을 언제 적용할지가 애매해지는 문제가 발생한다.

MSB를 자르고 LSB 5개를 반올림하는 연산은 결합법칙이 성립하지 않기 때문에 아무 시

점에서도 적용하면 Practice07에서의 FIR 필터 결과값과 달라질 수 있다. 따라서 round unit 적용 시점에 관한 문제와 추가적인 레지스터로 인한 면적 증가 문제를 고려하여 Direct Form을 이용하지 않고 Transposed Form 구조를 채택했다.



[Optimized Transposed Form Architecture]

$$A = X[n] \ll 2 - X[n]$$

$$B = X[n] \ll 8 + X[n] \ll 6$$

$$C = X[n] \ll 3 - X[n]$$

$$D = X[n]$$

위의 4가지 비트패턴을 만들기 위해 가산기를 3개 사용했고, 이를 이용해 각 tap별 연산결과를 만들고 round unit에 통과시킨 후 delay line을 거쳐서 결과가 출력되는 transposed fir filter의 구조를 위의 그림으로 나타냈다. 이렇게 하면 추가적인 레지스터도 필요하지 않고, 각 tap별로 연산이 진행되기 때문에 round unit의 적용도 지난 과제와 동일한 위치에서 할 수 있다. 혹시 모를 오버플로우를 방지하기 위해 round unit 전까지의 모든 값은 28비트로 부호확장해주었다.

[illegible]

테스트벤치를 수정하여 우리가 설계한 transposed form FIR filter만 출력값을 비교했다. 시뮬레이션이 완료된 시점에서 err_trans 값이 0이고, 시뮬레이션의 처음, 중간, 끝에서 메모리에 쓰여진 출력값과 정답이 일치하는 것을 볼 수 있다.

3. Synthesis Results

i. Timing

clock clk (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
X_N_reg[1]/CLK (dffs2)	0.00	0.00 r
X_N_reg[1]/Q (dffs2)	0.24	0.24 r
X_N_reg[1]/O0 (dffs2)	0.11	0.35 f
U169/Q (nnd2s2)	0.27	0.62 f
U395/Q (nnd2s2)	0.13	0.75 r
U170/Q (nnd2s3)	0.10	0.85 f
U677/Q (aoa1112s2)	0.46	1.31 r
U614/Q (nnd2s2)	0.18	1.49 f
U144/Q (nnd2s2)	0.18	1.57 r
U610/Q (nnd2s3)	0.12	1.79 f
U191/Q (i1s3)	0.08	1.87
U422/Q (aoa122s2)	0.18	2.05 f
U155/Q (xnr2s2)	0.36	2.41 f
add_1_root_add_26_2/A[10] (low_cost_transpose_fir_DW01_add_12)	0.00	
add_1_root_add_26_2/U136/Q (nnd2s2)	0.60	2.41 f
add_1_root_add_26_2/U149/Q (nnd3s3)	0.18	2.59 r
add_1_root_add_26_2/U147/Q (nnd2s3)	0.17	2.76 f
add_1_root_add_26_2/U174/Q (nnd2s3)	0.16	2.92 r
add_1_root_add_26_2/U172/Q (nnd3s3)	0.14	3.06 f
add_1_root_add_26_2/U89/OUTC (faddi1s3)	0.39	3.48
add_1_root_add_26_2/U57/OUTC (faddi1s3)	0.40	3.84
add_1_root_add_26_2/U174/Q (nnd2s2)	0.15	3.99 r
add_1_root_add_26_2/U177/Q (nnd3s3)	0.18	4.16 f
add_1_root_add_26_2/U150/Q (i1s2)	0.13	4.29 r
add_1_root_add_26_2/U122/Q (aoi21s3)	0.16	4.45 f
add_1_root_add_26_2/U106/Q (xnr2s2)	0.32	4.77 r
add_1_root_add_26_2/SUM[16] (low_cost_transpose_fir_DW01_add_12)	0.00	
add_0_root_add_26_2/B[16] (low_cost_transpose_fir_DW01_add_14)	0.00	4.77 r
add_0_root_add_26_2/U233/Q (nnd2s1)	0.00	4.77 r
add_0_root_add_26_2/U150/Q (i1s2)	0.17	4.94 f
add_0_root_add_26_2/U171/Q (aoi21s3)	0.13	5.07 f
add_0_root_add_26_2/U223/Q (nnd2s2)	0.14	5.21 f
add_0_root_add_26_2/U223/Q (nnd2s2)	0.18	5.39 r
add_0_root_add_26_2/U237/Q (aoi21s3)	0.26	5.64 f
add_0_root_add_26_2/U138/Q (i1s4)	0.20	5.85 r
add_0_root_add_26_2/U157/Q (aoi21s2)	0.26	6.11 f
add_0_root_add_26_2/U152/Q (xor2s2)	0.43	6.53 r
add_0_root_add_26_2/SUM[23] (low_cost_transpose_fir_DW01_add_14)	0.00	
R0/in[23] (round_unit_0)	0.00	6.53 r
R0/add_5/A[18] (round_unit_0_DW01_inc_0)	0.00	6.53 r
R0/add_5/U98/Q (nnd2s3)	0.11	6.64 f
R0/add_5/U98/Q (nnd2s3)	0.16	6.80 r
R0/add_5/U83/Q (nnd2s3)	0.12	6.92 f
R0/add_5/U94/Q (nnd2s2)	0.10	7.02 r
R0/add_5/U116/Q (xnr2s2)	0.28	7.30 r
R0/add_5/SUM[21] (round_unit_0_DW01_inc_0)	0.00	7.30 r
R0/U37/Q (nnd2s3)	0.09	7.40 f
R0/U36/Q (nnd2s3)	0.15	7.55 f
R0/out[21] (round_unit_0)	0.00	7.55 r
add_5/B[21] (low_cost_transpose_fir_DW01_add_2)	0.00	7.55 r
add_55/U196/Q (nnd2s1)	0.15	7.70 f
add_55/U182/Q (aoi21s2)	0.22	7.92 r
add_55/U217/Q (aoi21s3)	0.17	8.09 f
add_55/U216/Q (aoi21s3)	0.21	8.30 r
add_55/U215/Q (aoi21s3)	0.23	8.53 f
add_55/U210/Q (xor2s2)	0.39	8.92 r
add_55/SUM[23] (low_cost_transpose_fir_DW01_add_2)	0.00	8.92 r
U327/Q (nnd2s3)	0.09	9.01 f
U328/Q (i1s3)	0.08	9.09 f
y_0_reg_reg[23]/DIN (dffs1)	0.00	9.09 r
data arrival time		9.09
clock clk (rise edge)	9.20	9.20
clock network delay (ideal)	0.00	9.20
y_0_reg_reg[23]/CLK (dffs1)	0.00	9.20 r
library setup time	-0.30	8.90
data required time		8.90
data arrival time		8.90
data arrival time		-9.09
slack (VIOLATED)		-0.19

[Timing Report @ 9.2ns Period]

클럭 주기를 9.2ns로 합성했을 때는 음수 슬랙으로 타이밍 조건을 만족하지 못한다.

```

Startpoint: X_N_reg[1] (rising edge-triggered flip-flop clocked by clk)
Endpoint: y_1_reg[22] (rising edge-triggered flip-flop clocked by clk)
Path Group: clk
Path Type: max

```

Point	Incr	Path
clock clk (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
X_N_reg[1]/CLK (dffs2)	0.00	0.00 r
X_N_reg[1]/Q (dffs2)	0.27	0.27 r
U218/Q (nbs2)	0.32	0.59 r
U143/Q (t1s3)	0.12	0.71 f
U179/Q (nnd2s2)	0.14	0.85 r
U686/Q (nnd2s2)	0.11	0.96 f
U415/Q (aoai12s3)	0.37	1.33 r
U525/Q (xnr2s3)	0.48	1.82 f
U492/Q (xor2s3)	0.41	2.23 r
sub_0_root_sub_0_root_sub_27_3/A[7] (low_cost_transpose_fir_DW01_sub_0)	0.00	2.23 r
sub_0_root_sub_0_root_sub_27_3/U218/Q (or2s3)	0.22	2.44 r
sub_0_root_sub_0_root_sub_27_3/U217/Q (nnd2s3)	0.08	2.53 f
sub_0_root_sub_0_root_sub_27_3/U221/Q (xor2s2)	0.30	2.83 f
sub_0_root_sub_0_root_sub_27_3/DIFF[7] (low_cost_transpose_fir_DW01_sub_0)	0.00	2.83 f
R1/in[7] (round_unit_4)	0.00	2.83 f
R1/add_5/A[2] (round_unit_4_DW01_inc_0)	0.00	2.83 f
R1/add_5/U94/Q (t1s3)	0.09	2.92 r
R1/add_5/U95/Q (nor2s1)	0.24	3.16 f
R1/add_5/U104/Q (xor2s2)	0.40	3.55 r
R1/add_5/SUM[3] (round_unit_4_DW01_inc_0)	0.00	3.55 r
R1/U3/Q (mx21s2)	0.40	3.95 r
R1/out[3] (round_unit_4)	0.00	3.95 r
add_54/B[3] (low_cost_transpose_fir_DW01_add_3)	0.00	3.95 r
add_54/U258/Q (nnd2s2)	0.15	4.10 f
add_54/U133/Q (nnd3s3)	0.19	4.29 r
add_54/U148/Q (nnd2s2)	0.15	4.44 f
add_54/U151/Q (nnd3s3)	0.20	4.64 r
add_54/U137/Q (nnd2s2)	0.15	4.78 f
add_54/U168/Q (nnd3s3)	0.17	4.95 r
add_54/U193/OUTC (fadd1s3)	0.33	5.29 r
add_54/U192/OUTC (fadd1s3)	0.35	5.63 r
add_54/U181/OUTC (fadd1s3)	0.31	5.94 r
add_54/U180/OUTC (fadd1s3)	0.31	6.25 r
add_54/U187/Q (t1s3)	0.00	6.25 f
add_54/U199/Q (aoi21s3)	0.20	6.53 r
add_54/U198/Q (aoi21s3)	0.26	6.79 f
add_54/U201/Q (aoi21s3)	0.24	7.03 r
add_54/U200/Q (aoi21s3)	0.25	7.28 f
add_54/U174/Q (aoi21s3)	0.22	7.50 r
add_54/U202/Q (aoi21s3)	0.24	7.74 f
add_54/U162/Q (aoi21s2)	0.25	7.99 r
add_54/U167/Q (aoi21s3)	0.21	8.19 f
add_54/U169/Q (aoi22s2)	0.21	8.40 r
add_54/U185/Q (xnr2s2)	0.40	8.81 f
add_54/SUM[22] (low_cost_transpose_fir_DW01_add_3)	0.00	8.81 f
U409/Q (nnd2s3)	0.11	8.92 r
U139/Q (t1s4)	0.07	8.99 f
y_1_reg[22]/DIN (dffs1)	0.00	8.99 f
data arrival time		8.99
clock clk (rise edge)	9.30	9.30
clock network delay (ideal)	0.00	9.30
y_1_reg[22]/CLK (dffs1)	0.00	9.30 r
library setup time	-0.31	8.99
data required time		8.99
data arrival time		-8.99
slack (MET)		0.00

[Timing Report @ 9.3ns Period]

9.3ns 주기로 합성을 했을 때는 슬랙이 0이 되면서 타이밍 조건을 만족할 수 있게 됐다. 이 회로의 주파수는 약 107.5MHz이다. 리포트를 자세히 보면 X_N_reg를 통과한 신호가 여러 combinational 회로를 거친 뒤 adder/subtractor을 지나, round unit을 통과한 후 다시 adder를 지나 y_1_reg를 통과하는 경로가 크리티컬 패스임을 알 수 있다.

ii. Area

***** Report : area Design : low_cost_transpose_fir Version: Z-2007.03-SP4 Date : Tue May 12 16:47:49 2026 *****	***** Report : area Design : low_cost_transpose_fir Version: Z-2007.03-SP4 Date : Tue May 12 16:49:33 2026 *****
Library(s) Used: lec25dsc25_SS (File: /home/admin/lib/lec25/lec25dsc25_SS.db)	Library(s) Used: lec25dsc25_SS (File: /home/admin/lib/lec25/lec25dsc25_SS.db)
Number of ports: 40 Number of nets: 1253 Number of cells: 760 Number of references: 69	Number of ports: 40 Number of nets: 1237 Number of cells: 744 Number of references: 68
Combinational area: 159940.932049 Noncombinational area: 20686.275391 Net Interconnect area: undefined (No wire load specified)	Combinational area: 156930.061920 Noncombinational area: 20653.099365 Net Interconnect area: undefined (No wire load specified)
Total cell area: 180627.203125 Total area: undefined	Total cell area: 177583.156250 Total area: undefined

[Area Report @ 9.2ns Period vs 9.3ns Period]

클럭 주기가 9.2ns일 때 면적은 약 $180627\mu\text{m}^2$, 9.3ns일 때는 $177583\mu\text{m}^2$ 로 합성이 됐다. CPD를 줄이기 위해 DC가 면적을 늘리며 최적화를 시도한 것을 볼 수 있다.

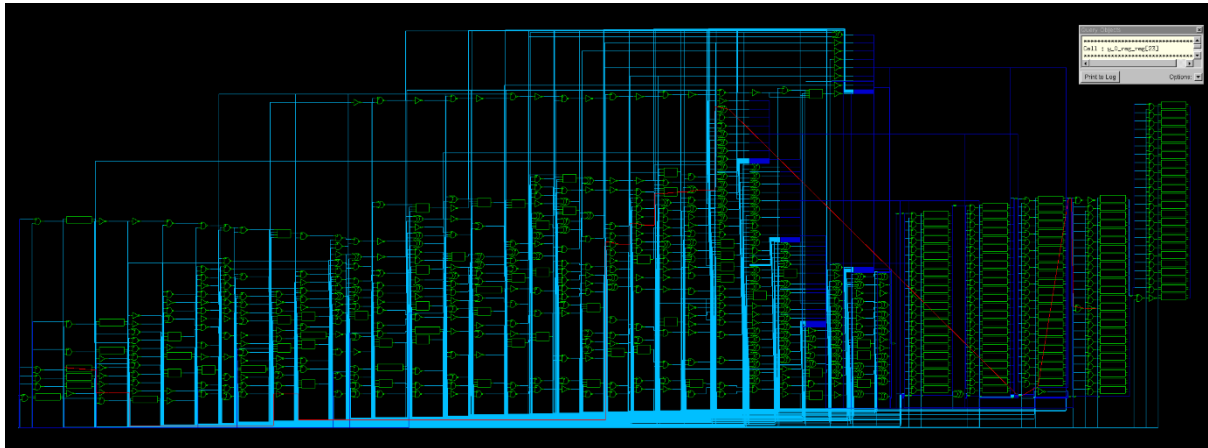
iii. Power

***** Report : power -analysis_effort low Design : low_cost_transpose_fir Version: Z-2007.03-SP4 Date : Tue May 12 16:47:50 2026 *****	***** Report : power -analysis_effort low Design : low_cost_transpose_fir Version: Z-2007.03-SP4 Date : Tue May 12 16:49:34 2026 *****
Library(s) Used: lec25dsc25_SS (File: /home/admin/lib/lec25/lec25dsc25_SS.db)	Library(s) Used: lec25dsc25_SS (File: /home/admin/lib/lec25/lec25dsc25_SS.db)
Operating Conditions: nom_pvt Library: lec25dsc25_SS Wire Load Model Mode: top	Operating Conditions: nom_pvt Library: lec25dsc25_SS Wire Load Model Mode: top
Global Operating Voltage = 2.25 Power-specific unit information : Voltage Units = 1V Capacitance Units = 1.000000pf Time Units = 1ns Dynamic Power Units = 1mW (derived from V,C,T units) Leakage Power Units = 1pW	Global Operating Voltage = 2.25 Power-specific unit information : Voltage Units = 1V Capacitance Units = 1.000000pf Time Units = 1ns Dynamic Power Units = 1mW (derived from V,C,T units) Leakage Power Units = 1pW
Cell Internal Power = 10.4114 mW (66%) Net Switching Power = 5.2847 mW (34%) Total Dynamic Power = 15.6961 mW (100%) Cell Leakage Power = 712.8483 uW	Cell Internal Power = 9.9089 mW (66%) Net Switching Power = 5.0287 mW (34%) Total Dynamic Power = 14.9376 mW (100%) Cell Leakage Power = 496.6480 uW

[Power Report @ 9.2ns Period vs 9.3ns Period]

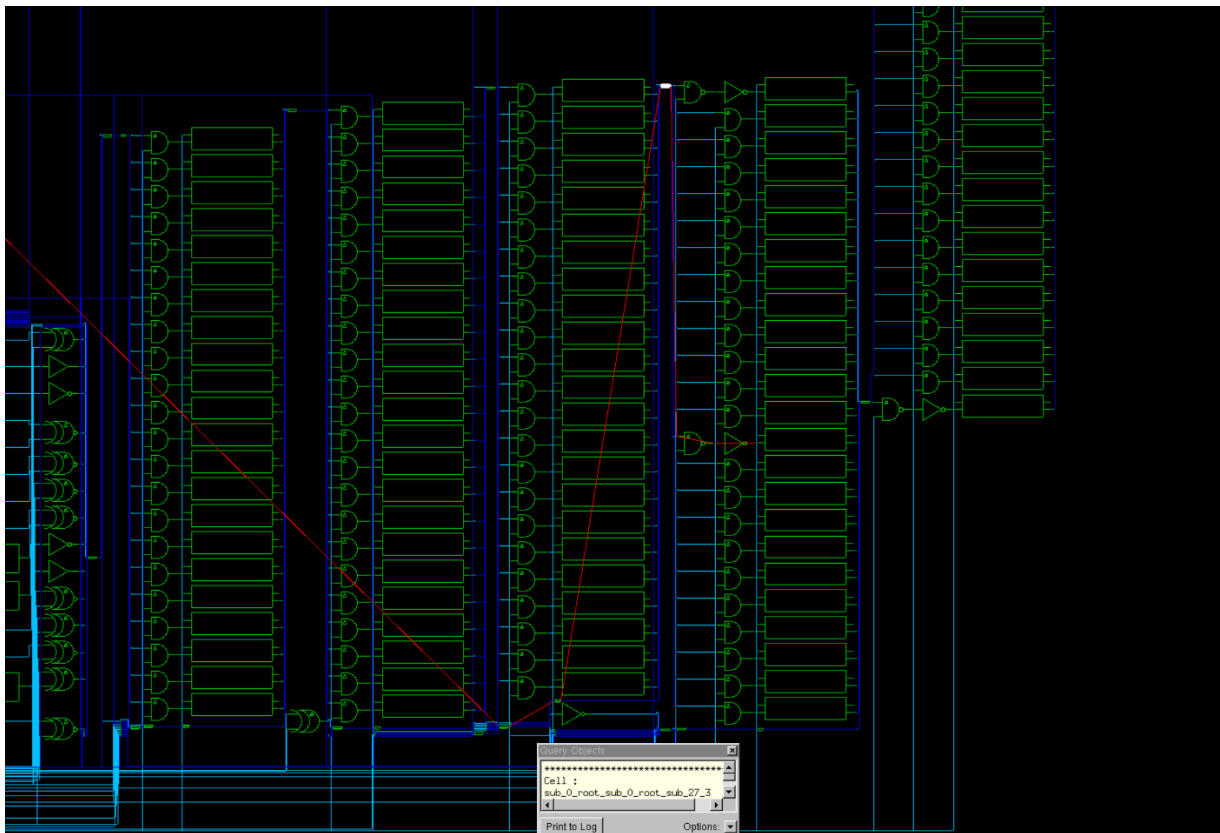
9.3ns 주기에서는 14.9376mW, 9.2ns에서는 15.6961mW의 동적 전력이 보고됐다. 클럭 주파수가 커지면서 소모 전력이 많아진 것을 확인할 수 있다.

iv. Schematics

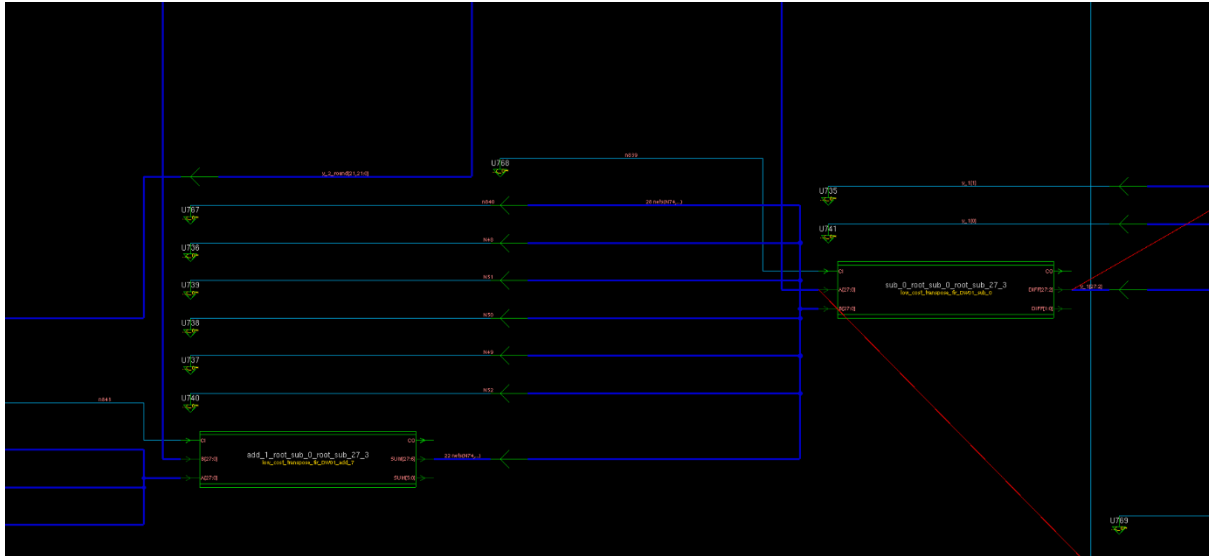


[Top Module with the Critical Path Highlighted]

위 사진은 탑모듈의 합성 후 스키매틱이고, 크리티컬 패스가 빨간색으로 표시되어 있다. 스키매틱의 좌측에는 X_N_reg가 위치해있고, 우측에 있는 22-bit, 23-bit, 24-bit 레지스터는 좌측부터 차례대로 y_4_reg, y_3_reg, y_2_reg, y_1_reg, y_0_reg이다. 양측의 레지스터 사이에 있는 큰 조합논리 회로는 공통 비트패턴 A, B, C, D를 만들기 위한 가산기이다.



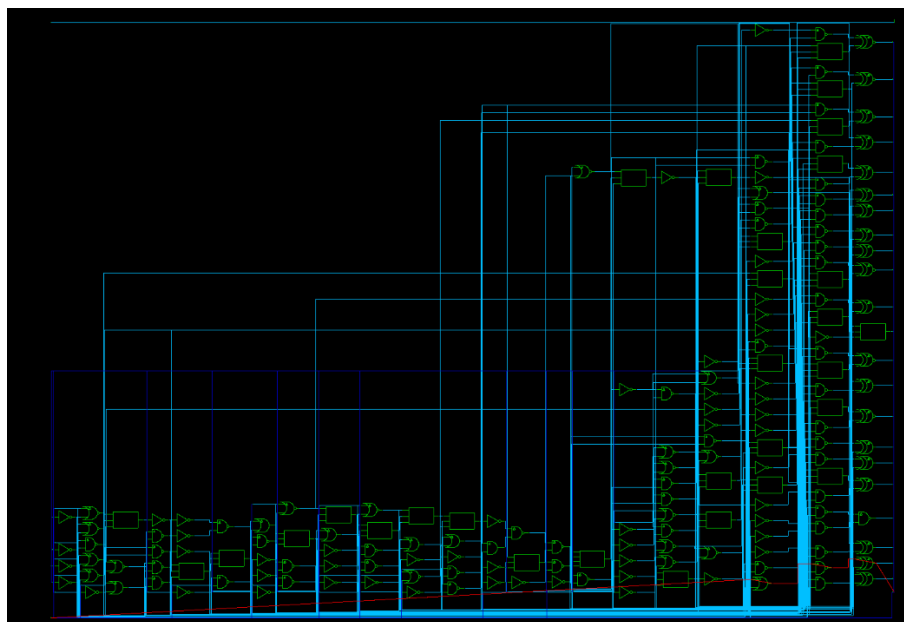
[Zoom in on the Highlighted Critical Path]



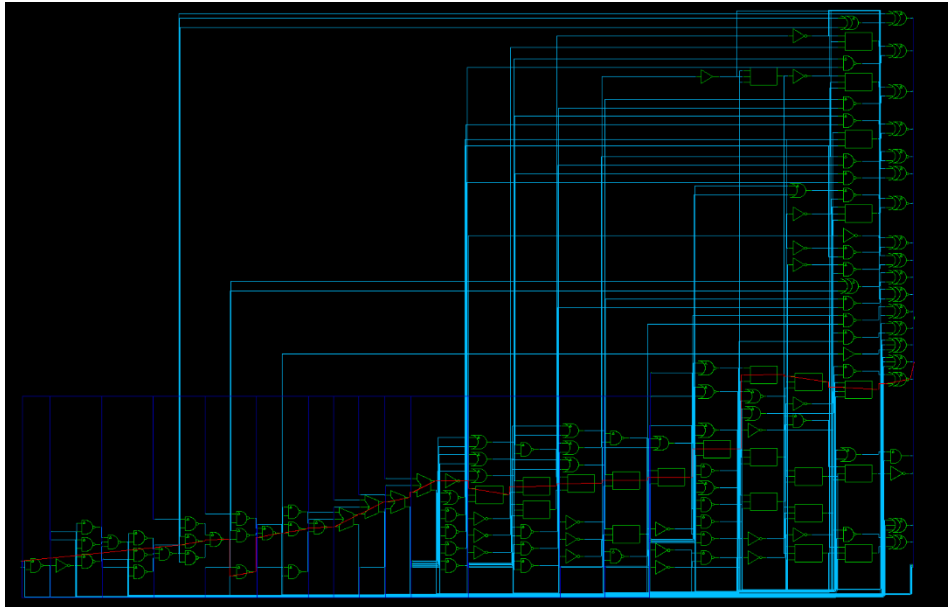
[Zoom in on the Highlighted Critical Path]

위 두 사진은 탑 모듈의 크리티컬 패스를 확대한 것으로, 두번째 사진은 첫번째 사진의 크리티컬 패스 중앙 부분을 확대한 것이다. 이 두번째 사진을 보면 크리티컬 패스 위에 놓여 있는 소자가 28비트 감산기로 구현되어 있고, 그 왼쪽에는 28비트 가산기가 있다. 탑모듈에서 계산된 공통 비트 패턴을 이용해 덧셈과 뺄셈을 하는 블록인 것으로 보인다.

다시 첫번째 사진을 보면 28비트 가산/감산 결과가 round unit을 통과하여 22비트가 되고, y_2_reg와 y_1_reg 사이의 가산기에 입력으로 들어가 y_2_reg의 출력과 더해진 뒤, y_1_reg에 입력되는 것을 볼 수 있다.

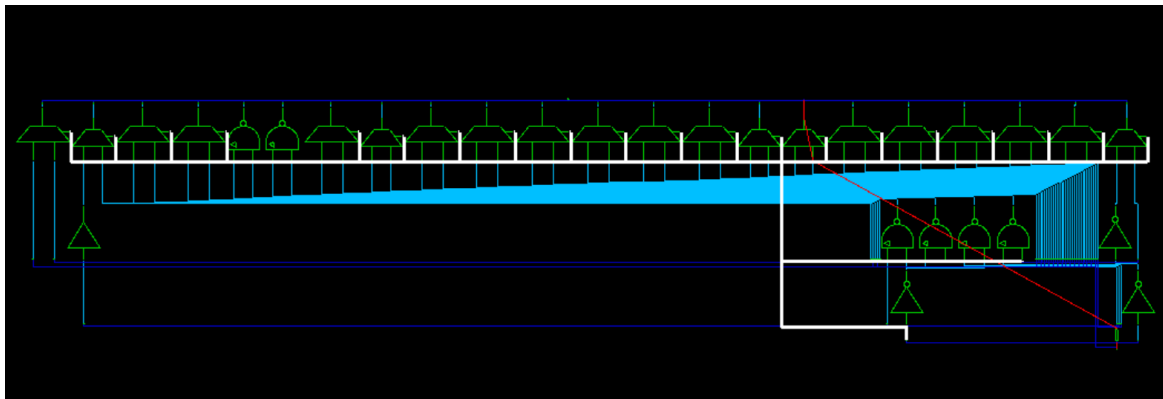


[Pre-round Adder]



[Post-round Adder]

위의 두 사진은 각각 round unit 이전에 있는 가산기 블록과 round unit 적용 후 레지스터 사이에 있는 가산기 블록의 합성 결과이다.



[Round Unit, Rotated by 90 Degrees]

위 그림은 라운드 유닛의 합성 결과로, 지난 과제와 비슷하게 여러 개의 멀티플렉서처럼 보이는 셀들의 집합으로 합성됐다. 흰색 선이 in[4]로, 입력의 5번째 비트인데, 이 비트가 멀티플렉서처럼 보이는 셀의 입력으로 연결되어 있다.

4. Discussion

i. Critical Path Analysis

스키매틱 항목에서 분석했던 것처럼 y_1_reg와 y_2_reg 사이에 있는 애더가 크리티컬 패스에 놓여 있었다. 이번 설계에서는 비트패턴 A, B, C, D 중에 B가 가장 계산하는 데

오래 걸릴 것이라고 추측해 볼 수 있다. 이는 B가 6만큼 left shift하고, 8만큼 left shift한 값 두 개를 더해야 해서 가산기 구현에 필요한 비트수가 제일 길기 때문이다. 하지만 A, B, C, D 모두 별도의 가산기 블록 없이 tap 모듈에서 조합논리 회로로 합성됐기 때문에 어떤 비트패턴이 계산하는데 가장 오래 걸리는지 정확하게 알기는 어렵다. 그래도 B가 계산하는데 제일 오래 걸린다고 가정하면, 스키매틱에서 봤던 크리티컬 패스가 설명이 된다. 이는 y_1_reg와 y_2_reg 사이에 들어갈 값을 계산하는데 B가 필요하기 때문이다.

한편 round unit 직전에는 세 개의 비트패턴을 더해줘야 하기 때문에 모든 tap마다 세 개의 가산기가 필요하고, 모든 wire가 28비트이기 때문에 대부분의 tap들이 비슷한 지연을 가질 것이라고 분석할 수 있다. 그래서 이 설계는 타이밍 경로들이 CPD 근처에 많이 분포할 것으로 추론할 수 있다. 이런 분포는 실제 칩을 만들 때 process variation의 영향에 취약하다.

ii. PPA Comparison

	Transposed	Low Cost Transposed
Clock	8.0ns (125MHz)	9.3ns (107.5MHz)
Area	385324 μm^2	177583 μm^2
Power	43.2149mW	14.9376mW177583

동작 속도 측면에서는 Low Cost가 일반 transposed보다 14% 느린 주파수를 보여주고 있다. 일반 Transposed Form은 하나의 14비트 곱셈기가 크리티컬 패스를 결정하는 반면, low cost transposed 구조는 비트패턴을 만들기 위한 조합논리 회로(가산기), 28비트 가산기, 그리고 22 (또는 23비트 또는 24비트) 가산기가 크리티컬 패스에 있기 때문에 더 느린 클럭으로 합성된 것으로 보인다.

면적은 low cost 설계를 적용했을 때 무려 54% 감소했다. 이는 두 구조 간 FF 사용량 차이와 가산기 합성 결과의 차이에 의한 것으로 보인다. 지난 transposed form에서는 계수가 정해져 있지 않아서 14비트 곱셈기를 5개 사용했고, 이를 저장하기 위한 14비트 레지스터도 5개 사용했던 반면, low cost 설계에서는 계수를 저장하기 위한 레지스터와 곱셈기를 모두 없애고, 오직 가산기만으로 기능을 구현할 수 있어 면적이 많이 감소했을 것이다.

전력 소모량은 더 큰 차이를 보였는데, low cost 설계 적용 시 65%의 전력량 감소를 보였다. 클럭 속도 감소로 인해 예상되는 소모 전력 감소보다 훨씬 큰 폭으로 줄어든 것을 확인할 수 있다. 면적과 마찬가지로 계수 레지스터를 구동하기 위해 필요했던 전력을 더 이상 쓰지 않게 됐고, 가산기와 shift만으로 곱셈을 대체할 수 있게 되어 더 적은 셀을 사용할 수 있어 전력도 크게 줄어든 것을 확인할 수 있다.

결론적으로 low cost 설계를 통해 속도 측면에서 약간의 손해가 있었지만, 면적과 전력을 모두 50% 이상 절감할 수 있었다. 클럭 속도가 줄었다는 점에서 low cost 설계 역시 만능 설계는 아니라는 것을 알 수 있지만, 속도와 면적, 파워 사이의 tradeoff를 잘 이해하고 회로가 쓰이는 도메인에 맞게 설계를 하는 능력이 필요할 것이다. 필터 계수가 고정된 edge device에서 사용이 필요할 경우 면적과 전력 소모량을 줄이기 위해 low cost 설계를 적용할 수 있을 것이다.

iii. Verilog

CSD로 바꾼 후부터는 더 이상 2's complement로 해석을 할 수 없다. 이런 원리에서 MSB가 1인 숫자를 0 (-1)로 바꿀 수 있었고, MSB가 0인 숫자를 10(-1)로 바꿀 수 있었다. 하지만 이 ternary digit에 곱해지는 $X[n]$ 은 2's complement signed 형식이기 때문에 베릴로그 코드를 짤 때 이에 유의해야 한다. 또한 혹시 모를 비트너비 실수나 오버플로우를 방지하기 위해 round unit 전까지의 모든 wire를 28비트로 설정했는데 이 wire에 assign되는 값들은 28비트에 미치지 않기 때문에 DC가 컴파일할 때 자동으로 sign extend를 해줘야 한다. 따라서 모든 wire와 reg를 signed로 선언해 sign extension이 제대로 일어날 수 있게 했다.

iv. FIR Filter Characteristics

M-tap FIR 필터의 n번째 출력은 아래의 공식을 따른다.

$$y(n) = \sum_{k=0}^{M-1} h(k) x(n-k)$$

즉, 5-tap FIR filter에서 $y[0] \sim y[3]$ 까지의 값은 존재하지 않는 과거의 입력신호를 이용한 결과이므로 유효하지 않은 값이다. 실제로 유효한 출력은 $y[4] \sim y[255]$ 까지이므로 테스트 벤치에서 확인하는 결과는 252개이다. 입력 벡터는 256개이지만 출력 결과는 4개 적은 것이다.